

Generative Modeling: VAEs, GANs, Score Matching, and Diffusion

Mathematics of Deep Learning — Chapter 18

Yin Yang

Foundation Books Project

Roadmap

Distribution Learning

Latent Variables and Decoders

Variational Autoencoders

Generative Adversarial Networks

Score Functions and Denoising

The Diffusion Forward Process

The Learned Reverse Process

Conditional Generation and Guidance

Evaluation and Diagnostics

Summary

Distribution Learning

From Classification to Generation

Previous vision chapters: given image x , predict label y .

- Classifier estimates a conditional $p(y \mid x)$.
- A *generator* models, approximates, or samples from $p(x)$ or $p(x \mid c)$.
- Running example: tiny 8×8 grayscale symbol images, $x \in [0, 1]^{64}$, training set $\mathcal{D} = \{x_1, \dots, x_m\}$.

Four mathematical routes lead to the same applied question: reconstruction (VAE), discrimination (GAN), score estimation, denoising (diffusion).

Generation is distribution learning plus a sampling rule.

Generative Training and Sampling Loop

training images

$$x \sim p_{\text{data}}$$

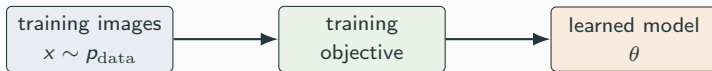
(1) Training images come from an unknown p_{data} .

Generative Training and Sampling Loop



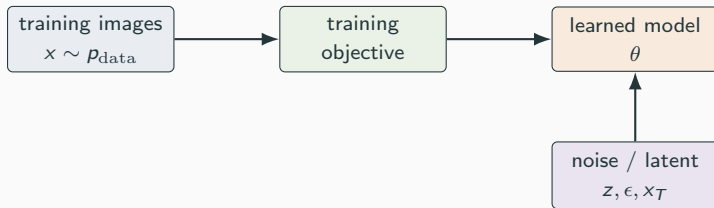
- (1) Training images come from an unknown p_{data} .
- (2) An objective scores how well the model matches data.

Generative Training and Sampling Loop



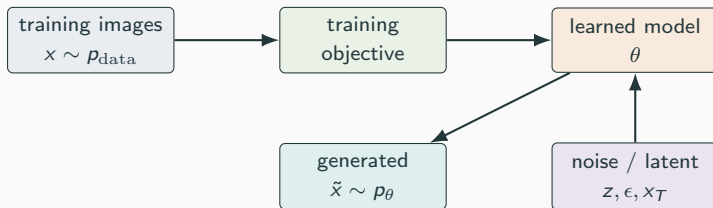
- (1) Training images come from an unknown p_{data} .
- (2) An objective scores how well the model matches data.
- (3) Training updates parameters θ .

Generative Training and Sampling Loop



- (1) Training images come from an unknown p_{data} .
- (2) An objective scores how well the model matches data.
- (3) Training updates parameters θ .
- (4) Sampling supplies noise or latent input.

Generative Training and Sampling Loop



- (1) Training images come from an unknown p_{data} .
- (2) An objective scores how well the model matches data.
- (3) Training updates parameters θ .
- (4) Sampling supplies noise or latent input.
- (5) The model returns a fresh image-like sample.

The chapter studies both pieces: the training objective and the sampling rule.

Modes and Coverage

Suppose p_{data} puts probability $1/4$ on each of four symbol types. Generator places probabilities $(0.70, 0.20, 0.10, 0.00)$.

The fourth mode disappeared. Total-variation distance is then bounded:

$$d_{\text{TV}}(p_{\text{data}}, p_{\theta}) \geq |1/4 - 0| = 0.25.$$

- Mode collapse: probability on a few modes, near-zero elsewhere.
- Sharp individual samples can still be a wrong distribution.
- A sample gallery hides missing modes; counts do not.

Sample quality is not the same as distribution quality.

Four Families, Four Mathematical Objects

Family	What it learns directly	Applied check
VAE	lower bound on $\log p_\theta(x)$	reconstructions, KL
GAN	generator/discriminator game	realism, coverage
Score model	$\nabla_x \log p(x)$ estimate	direction field
Diffusion	learned reverse noising	noise-level loss, samples

- Likelihood-based models score a density.
- Implicit models supply samples but no easy density.
- Score / diffusion learn gradients of log-density.

The right diagnostic depends on what mathematical object the family exposes.

Latent Variables and Decoders

Latent variable

A hidden coordinate $z \in \mathbb{R}^k$, sampled from a prior $p(z)$. Not observed in the dataset.

Decoder

Deterministic form: $\tilde{x} = g_\theta(z)$, $z \sim p(z)$. Probabilistic form: $\tilde{x} \sim p_\theta(x | z)$.

Standard prior: $z \sim \mathcal{N}(0, I_k)$. Pushforward distribution: law of $\tilde{x} = g_\theta(z)$.

Latent variables are modeling coordinates, not labels in the dataset.

Affine Decoder Moments

If $z \sim \mathcal{N}(0, I_k)$ and $\tilde{x} = Az + b$ with $A \in \mathbb{R}^{d \times k}$, $b \in \mathbb{R}^d$:

$$\mathbb{E}[\tilde{x}] = b, \quad \text{Cov}(\tilde{x}) = AA^\top.$$

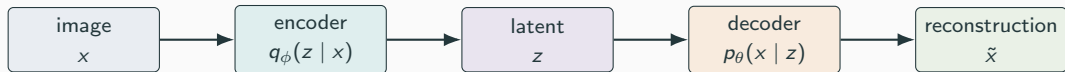
- b : center of generated distribution (mean appearance).
- $AA^\top$: spread and correlations (sample diversity).
- Real decoders are nonlinear; same dependency pattern.

Latent interpolation as a diagnostic: smooth paths in z should produce smooth changes in $\tilde{x}$.

Changing the prior changes which codes are visited; changing the decoder changes how codes become images.

Variational Autoencoders

VAE Components



Exact posterior $p_\theta(z | x)$ requires $p_\theta(x) = \int p_\theta(x | z)p(z) dz$ — usually intractable.

- $q_\phi(z | x)$: approximate posterior (encoder).
- $p_\theta(x | z)$: decoder likelihood.
- $p(z) = \mathcal{N}(0, I)$: shared prior.

Train an encoder for inference and a decoder for generation.

The Evidence Lower Bound (ELBO)

$$\mathcal{L}_{\text{ELBO}}(\theta, \phi; x) = \mathbb{E}_{z \sim q_{\phi}(\cdot | x)}[\log p_{\theta}(x | z)] - D_{\text{KL}}(q_{\phi}(z | x) \| p(z)).$$

Identity:

$$\log p_{\theta}(x) = \mathcal{L}_{\text{ELBO}}(\theta, \phi; x) + D_{\text{KL}}(q_{\phi}(z | x) \| p_{\theta}(z | x)).$$

Since $\text{KL} \geq 0$, $\mathcal{L}_{\text{ELBO}}$ is a lower bound on $\log p_{\theta}(x)$.

- Reward: reconstruction log-likelihood (large is good).
- Penalty: KL between encoder and prior (small is good).
- Bound is tight when $q_{\phi}(z | x) = p_{\theta}(z | x)$.

Training raises a computable lower bound; the gap is a KL to the exact posterior.

ELBO — One-Example Calculation

Let $q(z | x) = \mathcal{N}(\mu, \sigma^2)$, $p(z) = \mathcal{N}(0, 1)$ in 1D:

$$D_{\text{KL}}(q \| p) = \frac{1}{2}(\mu^2 + \sigma^2 - \log \sigma^2 - 1).$$

With $\mu = 1$, $\sigma^2 = 2$:

$$D_{\text{KL}} = \frac{1}{2}(1 + 2 - \log 2 - 1) \approx 0.653.$$

If reconstruction term = -3.2 :

$$\mathcal{L}_{\text{ELBO}} = -3.2 - 0.653 \approx -3.853.$$

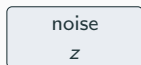
Reparameterization for gradients through sampling: with $\eta \sim \mathcal{N}(0, 1)$,

$$z = \mu_{\phi}(x) + \sigma_{\phi}(x)\eta.$$

Reparameterization moves randomness off the parameter path so pathwise gradients flow.

Generative Adversarial Networks

GAN: Two-Player Game



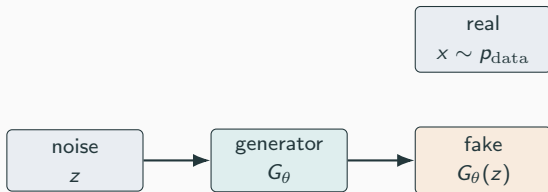
(1) Noise enters.

GAN: Two-Player Game



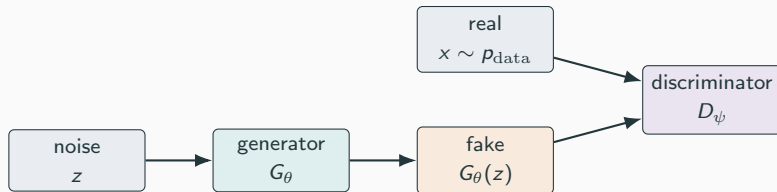
(1) Noise enters. (2) Generator maps noise to image.

GAN: Two-Player Game



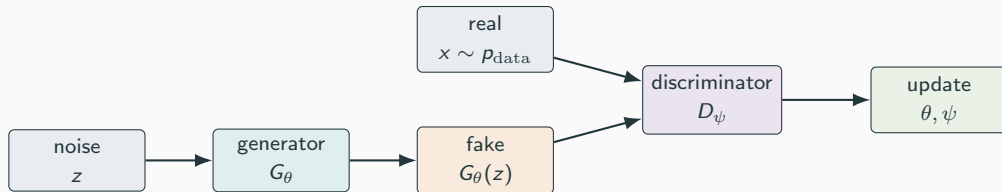
- (1) Noise enters. (2) Generator maps noise to image.
(3) Discriminator sees both fake and real.

GAN: Two-Player Game



- (1) Noise enters. (2) Generator maps noise to image.
- (3) Discriminator sees both fake and real.
- (4) It scores realism in $(0, 1)$.

GAN: Two-Player Game



$$\min_{\theta} \max_{\psi} \mathbb{E}_{x \sim p_{\text{data}}} [\log D_{\psi}(x)] + \mathbb{E}_{z \sim p(z)} [\log(1 - D_{\psi}(G_{\theta}(z)))].$$

(1) Noise enters. (2) Generator maps noise to image.

(3) Discriminator sees both fake and real.

(4) It scores realism in (0, 1). (5) Both players update.

No fixed reconstruction loss — realism is a learned signal.

Optimal Discriminator and JS

For fixed G with generator density p_g , the discriminator maximizer is

$$D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}.$$

Substituting back, the value of the minimax game becomes

$$-\log 4 + 2 \text{JS}(p_{\text{data}} \parallel p_g),$$

minimized exactly when $p_g = p_{\text{data}}$.

- Implicit generative model: easy to sample, hard to evaluate $p_\theta(x)$.
- Saturating gradients motivate non-saturating loss $-\log D_\psi(G_\theta(z))$.
- WGAN: $\min_\theta \max_{\text{Lip}(f_\psi) \leq 1} \mathbb{E}_{p_{\text{data}}}[f_\psi] - \mathbb{E}_{p_g}[f_\psi]$.

Different GAN losses change pressure on the same fixed point $p_g = p_{\text{data}}$.

Score Functions and Denoising

Definition

For a differentiable positive density $p(x)$ on $\mathbb{R}^d$,

$$s(x) = \nabla_x \log p(x) \in \mathbb{R}^d.$$

A direction field, not a probability.

Gaussian case: $p(x) = \mathcal{N}(x; \mu, \sigma^2)$ gives

$$\nabla_x \log p(x) = -\frac{x - \mu}{\sigma^2}.$$

For $\mathcal{N}(0, 4)$ at $x = 2$: score = -0.5 (points left, toward $\mu = 0$).

The score points back toward higher density at the current location.

Score Field Around a Gaussian Mean



(1) Sample noisy points scattered along the axis.

Score Field Around a Gaussian Mean



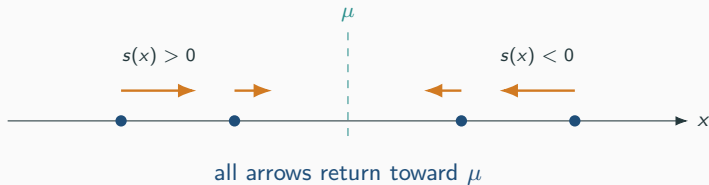
- (1) Sample noisy points scattered along the axis.
- (2) Score at each point gives a direction vector.

Score Field Around a Gaussian Mean



- (1) Sample noisy points scattered along the axis.
- (2) Score at each point gives a direction vector.
- (3) Signs flip across the mean.

Score Field Around a Gaussian Mean



- (1) Sample noisy points scattered along the axis.
- (2) Score at each point gives a direction vector.
- (3) Signs flip across the mean.
- (4) The field collectively pulls samples toward the data region.

In high dimensions, score models learn an analogous direction field over noisy images.

Denoising Connects to Score

Forward corruption:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$

Train $\epsilon_\theta(x_t, t)$ to predict the added noise via squared error:

$$\mathbb{E}[\|\epsilon - \epsilon_\theta(x_t, t)\|_2^2].$$

Among all $f(x_t, t)$, the minimizer is the conditional mean $\mathbb{E}[\epsilon \mid x_t]$. This connects to the marginal score:

$$\nabla_{x_t} \log q_t(x_t) = -\frac{\mathbb{E}[\epsilon \mid x_t]}{\sqrt{1 - \bar{\alpha}_t}}.$$

Noise prediction is regression; the regression target induces a score estimate.

The Diffusion Forward Process

Single step: $q(x_t | x_{t-1}) = \mathcal{N}(\sqrt{1 - \beta_t} x_{t-1}, \beta_t I)$, $\beta_t \in (0, 1)$. With $\alpha_t = 1 - \beta_t$ and

$$\bar{\alpha}_t = \prod_{s \leq t} \alpha_s:$$

$$q(x_t | x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) I),$$

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon.$$

Numerical example: $x_0 = 1$, $\bar{\alpha}_t = 0.64$, $\epsilon = -0.5$:

$$x_t = 0.8 \cdot 1 + 0.6 \cdot (-0.5) = 0.5.$$

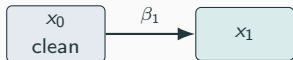
Signal-to-noise: $\text{SNR}(t) = \bar{\alpha}_t / (1 - \bar{\alpha}_t)$.

The forward process is chosen, not learned; supervision uses the known ϵ .

x_0
clean

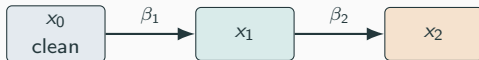
(1) Begin with a clean image x_0 .

Forward Trajectory, Built Up



- (1) Begin with a clean image x_0 .
- (2) Add a small Gaussian step — signal still dominant.

Forward Trajectory, Built Up



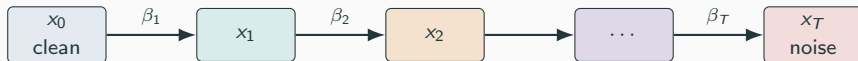
- (1) Begin with a clean image x_0 .
- (2) Add a small Gaussian step — signal still dominant.
- (3) Repeat; structure degrades gradually.

Forward Trajectory, Built Up



- (1) Begin with a clean image x_0 .
- (2) Add a small Gaussian step — signal still dominant.
- (3) Repeat; structure degrades gradually.
- (4) Many small steps drive $\bar{\alpha}_t$ toward zero.

Forward Trajectory, Built Up



- (1) Begin with a clean image x_0 .
- (2) Add a small Gaussian step — signal still dominant.
- (3) Repeat; structure degrades gradually.
- (4) Many small steps drive $\bar{\alpha}_t$ toward zero.
- (5) At time T , x_T is nearly pure Gaussian noise.

The schedule decides how hard each later recovery step has to be.

The Learned Reverse Process

From Noise Prediction to $\hat{x}_0$

Network: $\epsilon_\theta(x_t, t)$, trained by

$$\mathbb{E}_{x_0, \epsilon, t} [\|\epsilon - \epsilon_\theta(x_t, t)\|_2^2].$$

Solve the forward equation for the clean state:

$$\hat{x}_0(x_t, t) = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}.$$

Scalar example: $\bar{\alpha}_t = 0.64$, $x_t = 0.20$, $\epsilon_\theta = -0.75$:

$$\hat{x}_0 = \frac{0.20 - 0.6(-0.75)}{0.8} = 0.8125.$$

The denoiser predicts the noise; algebra converts that into an estimated clean image.

DDPM Reverse Step

Forward posterior given x_0 :

$$q(x_{t-1} \mid x_t, x_0) = \mathcal{N}(\tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t I).$$

Substituting the model's $\hat{x}_0$ gives the sampler mean

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right).$$

Ancestral sampling:

$$x_T \sim \mathcal{N}(0, I), \quad x_{t-1} = \mu_\theta(x_t, t) + \sigma_t z, \quad z \sim \mathcal{N}(0, I).$$

The reverse template is Gaussian algebra; only the missing x_0 information is learned.

Reverse Trajectory, Step by Step

x_T
noise

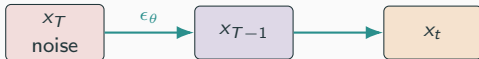
(1) Start from pure Gaussian noise.

Reverse Trajectory, Step by Step



- (1) Start from pure Gaussian noise.
- (2) Network predicts noise; one Gaussian reverse step.

Reverse Trajectory, Step by Step



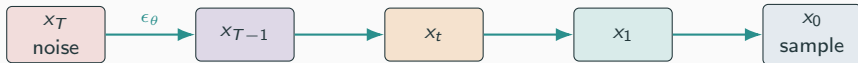
- (1) Start from pure Gaussian noise.
- (2) Network predicts noise; one Gaussian reverse step.
- (3) Repeat across many noise levels.

Reverse Trajectory, Step by Step



- (1) Start from pure Gaussian noise.
- (2) Network predicts noise; one Gaussian reverse step.
- (3) Repeat across many noise levels.
- (4) Late steps refine fine structure.

Reverse Trajectory, Step by Step



- (1) Start from pure Gaussian noise.
- (2) Network predicts noise; one Gaussian reverse step.
- (3) Repeat across many noise levels.
- (4) Late steps refine fine structure.
- (5) Final state x_0 is the generated sample.

DDIM uses the same ϵ_θ on a shorter time grid — changes the sampler, not the training target.

Probability-Flow ODE and Score

Forward SDE:

$$dx = f(x, t) dt + g(t) dW_t.$$

Reverse SDE (informal):

$$dx = [f(x, t) - g(t)^2 \nabla_x \log p_t(x)] dt + g(t) d\bar{W}_t.$$

Probability-flow ODE (same marginals):

$$dx = [f(x, t) - \frac{1}{2}g(t)^2 \nabla_x \log p_t(x)] dt.$$

- Learned object: score field $\nabla_x \log p_t(x)$.
- Sampler: stochastic or deterministic numerical path.
- Denoiser biases at certain noise levels propagate as artifacts.

Denoiser and sampler are separable design choices.

Conditional Generation and Guidance

Conditional Score Decomposition

Target shifts from $p(x)$ to $p(x | c)$: class label, caption, mask, sketch, embedding. Conditional denoiser: $\epsilon_\theta(x_t, t, c)$. Bayes' rule gives, for the noisy-time density:

$$\nabla_x \log p_t(x | c) = \nabla_x \log p_t(x) + \nabla_x \log p_t(c | x).$$

- Classifier guidance: add classifier gradient $\nabla_x \log p_t(c | x)$ to the unconditional score.
- Classifier-free: train one denoiser to handle both c and the null condition $\emptyset$.

The condition-adherence direction is a separate gradient added to the data-distribution direction.

Classifier-Free Guidance

Define

$$\hat{\epsilon} = \epsilon_{\theta}(x_t, t, \emptyset) + w(\epsilon_{\theta}(x_t, t, c) - \epsilon_{\theta}(x_t, t, \emptyset)).$$

Worked example: $u = \epsilon_{\theta}(\cdot, \emptyset) = (2, 1)$, $v = \epsilon_{\theta}(\cdot, c) = (1, 3)$, $w = 2$:

$$\hat{\epsilon} = u + 2(v - u) = (0, 5).$$

Extrapolates *past* the conditional prediction.

- $w = 0$: unconditional. $w = 1$: conditional. $w > 1$: amplify.
- Larger w : better condition adherence, less diversity.
- Sweep w to expose the realism vs. control tradeoff.

Guidance trades diversity for condition adherence; report a sweep, not a single value.

Evaluation and Diagnostics

Failure Modes Map to Diagnostics

Failure mode	Symptom	Useful check
Poor sample quality	blurry / distorted images	sample grids
Poor coverage	missing types / repeats	class-count tables
Memorization	near-copies of training	nearest-neighbor
Bad conditioning	prompt ignored	condition sweep
Over-guidance	saturated / repetitive	guidance-scale sweep
Training mismatch	low loss, poor samples	per-noise-level loss

Tiny coverage example: 120 samples, 70/50/0 across three balanced classes. Balanced target would be ~ 40 per class.

No single metric detects every problem.

Diagnostics by Family

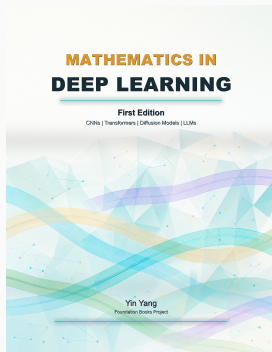
- **VAE.** Reconstructions, random samples, KL value. Small KL: latent ignored; large KL: irregular latent space.
- **GAN.** Realism by sample inspection, coverage by class counts, learned summaries (e.g. FID) as proxies, not proofs.
- **Score / diffusion.** Noise-prediction loss split by time t . High- t errors: trajectory starts wrong. Low- t errors: final-step artifacts remain.
- **Conditional.** Image quality *and* alignment with c — a clean loop is wrong for “vertical stroke”.

Match the diagnostic to the mathematical object the family exposes.

Summary

- **Generation** = distribution learning + sampling rule. A sample gallery is not a distribution proof.
- **VAE**. Maximize $\mathcal{L}_{\text{ELBO}}$ = reconstruction – KL to prior; reparameterization passes gradients through sampling.
- **GAN**. Minimax game; optimal D^* leads to JS; WGAN replaces it with a Lipschitz-constrained transport proxy.
- **Score**. Learn $\nabla_x \log p(x)$ as a direction field; denoising regression supplies the supervision signal.
- **Diffusion**. Fixed Gaussian forward $\rightarrow$ learned reverse via $\epsilon_\theta(x_t, t)$; DDPM, DDIM, SDE/ODE samplers all reuse the same denoiser.
- **Guidance**. Add $\nabla_x \log p_t(c | x)$ to the unconditional score; classifier-free implements it in one model.
- **Diagnostics**. Quality, coverage, memorization, conditioning, over-guidance, training mismatch — each needs its own check.

**Next: large-scale training systems, distributed optimization,
and serving pipelines for the generative models above.**



Mathematics in Deep Learning

Chapter overview slides for the book by Yin Yang.

Book page	foundation-books.github.io/mathematics-in-deep-learning
Code	github.com/foundation-books/mathematics-in-deep-learning-code
Paperback	amazon.com/dp/BOH1ZZHDT2
Hardcover	amazon.com/dp/BOH1ZL11MJ
Kindle	amazon.com/dp/B0GX32M8SP
License	CC BY-NC-ND 4.0

These free overview slides may be shared non-commercially with attribution and without modifications. Full explanations, exercises, and companion-code context are in the book and linked repository.